Pokédex — Bootcamp II (Etapa 03)

Aplicação web de uma **Pokédex** feita com HTML, CSS e JavaScript, que consome a

PokeAPI para exibir os Pokémon e usa o **Supabase** (PostgreSQL) para persistir

dados em um banco real. O projeto é conteinerizado com **Docker** e conta com

testes automatizados (Jest) e **Integração Contínua (GitHub Actions)**.

👥 Integrantes

Nome	Usuário GitHub	Função
Gabriel Paes	@GabrielPaes17	Desenvolvimento completo

🔗 Links do projeto

- **Repositório (GitHub):** https://github.com/GabrielPaes17/bootcamp2-app
- **Site publicado (GitHub Pages):** https://gabrielpaes17.github.io/bootcamp2-app/
- **Imagem no Docker Hub:** https://hub.docker.com/r/gabriel0paes17/bootcamp2-app

⚠ Confira se os dois links acima (GitHub Pages e Docker Hub) estão exatamente

corretos e ajuste se necessário.

📌 Sobre a aplicação

A Pokédex permite:

- Listar e visualizar Pokémon com seus dados (nome, imagem, tipos etc.);
- Salvar Pokémon favoritos em um banco de dados real;
- Listar e excluir os favoritos salvos (CRUD: criar / listar / excluir).

API utilizada — PokeAPI

Os dados dos Pokémon são obtidos em tempo real da PokeAPI,

uma API pública e gratuita. A aplicação faz requisições HTTP (fetch) para buscar

as informações e exibi-las na tela.

Banco de dados — Supabase (PostgreSQL)

A persistência é feita no **Supabase**, que usa **PostgreSQL** por baixo. A

aplicação se conecta via @supabase/supabase-js@2 usando a **chave pública (publishable / anon)**.

CRUD implementado: criar, listar e excluir favoritos.

⚠ **Limitação conhecida:** o acesso ao banco usa a chave pública (anon) com

RLS (Row Level Security) aberto para fins de aprendizado. Em um projeto

real, as regras de segurança (RLS) deveriam ser restritas. A chave usada é

sempre a **pública**, nunca a chave secreta.

🛠 Tecnologias

- **Front-end:** HTML, CSS, JavaScript
- **API:** PokeAPI
- **Banco de dados:** Supabase (PostgreSQL)
- **Conteinização:** Docker (imagem base nginx:alpine)
- **Testes:** Jest
- **CI/CD:** GitHub Actions

▶ Como rodar localmente

1. Clone o repositório:

   git clone https://github.com/GabrielPaes17/bootcamp2-app.git

   cd bootcamp2-app
2. Como o front-end é estático, você pode abrir o arquivo index.html direto no

   navegador, ou subir um servidor local simples:

   python3 -m http.server 8080
3. Acesse no navegador:

   http://localhost:8080

🐳 Como rodar com Docker

A imagem já está publicada no Docker Hub. Para rodar:

docker run -p 8080:80 gabriel0paes17/bootcamp2-app:latest

Depois acesse: http://localhost:8080

Tags disponíveis no Docker Hub: 1.0, 1.1, latest

Para rodar uma versão específica, troque latest pela tag desejada, por exemplo: gabriel0paes17/bootcamp2-app:1.1

🧪 Como rodar os testes

Os testes são escritos com **Jest**.

1. Instale as dependências:

   npm install

2. Rode os testes:

   npm test

Os testes validam as funções de validacao.js (validarEntrada e formatarNome).

☸ Integração Contínua (CI) e proteção da branch

O repositório usa **GitHub Actions** para rodar os testes automaticamente a cada

push e a cada Pull Request (workflow em .github/workflows/ci.yml, job testes).

A branch main está **protegida** com as seguintes regras:

- ☑ Exige Pull Request antes do merge (ninguém faz push direto na main);
- ☑ Exige 1 aprovação (revisão de código);
- ☑ Exige que o check testes (CI) passe antes do merge.

🖻 Evidências

1. CI rodando (verde)

O workflow de testes rodou com sucesso no GitHub Actions.



2. Teste de fogo — merge bloqueado pelo CI

Foi criado um Pull Request com um teste quebrado **de propósito**. O CI ficou

vermelho e o merge foi **bloqueado**, provando que a proteção funciona.



3. Conflito de merge

Foi provocado um conflito de merge (duas branches alterando o mesmo arquivo) e

em seguida **resolvido** manualmente.



Estrutura do projeto

```
bootcamp2-app/
├── index.html
├── (CSS / JS da aplicação)
├── validacao.js            # funções testadas
├── validacao.test.js       # testes (Jest)
├── package.json            # configuração do Jest
├── Dockerfile              # imagem Docker (nginx:alpine)
├── .dockerignore
├── .gitignore
└── .github/workflows/ci.yml  # Integração Contínua
```